

Exercícios Avaliativos — CRUD de Produtos

Implementação completa de um CRUD com Bun, Express e JavaScript no navegador

VALOR
4,0 pontos

DIFICULDADE
Média

FORMATO
Individual ou dupla

PRAZO
Até 01/06/2026

1. Contexto

Na aula do dia **11/05/2026** construímos juntos uma pequena aplicação web para listar produtos: um **backend** em Bun + Express que expõe a rota `GET /produtos` (com filtros por `nome`, `tipo` e `status`) e um **frontend** em HTML, CSS e JavaScript puro que consome essa API e renderiza uma tabela.

O código entregue em aula contempla apenas a operação de **leitura**. Esta avaliação propõe que vocês evoluam o projeto e implementem o **CRUD completo**: *Create, Read, Update e Delete*, tanto no servidor quanto na interface.

2. Objetivo

Implementar um CRUD completo de produtos, exercitando rotas HTTP, métodos REST, manipulação de arrays em JavaScript, comunicação entre frontend e backend via `fetch`, e manipulação do DOM para refletir as mudanças de estado.

3. Requisitos do Backend

Partindo do `server.js` construído em aula, vocês devem manter a rota existente e adicionar as demais operações REST. A persistência continua sendo **em memória** (no próprio array `produtos`) — não é necessário banco de dados nesta avaliação.

Método	Rota	Comportamento esperado
GET	<code>/produtos</code>	Lista os produtos. Já implementada em aula, deve manter os filtros por <code>nome</code> , <code>tipo</code> e <code>status</code> .
GET	<code>/produtos/:id</code>	Retorna um único produto pelo seu <code>id</code> . Se não existir, responder com status <code>404</code> .

Método	Rota	Comportamento esperado
POST	/produtos	Cria um novo produto a partir do corpo da requisição. O <code>id</code> deve ser gerado automaticamente pelo servidor. Responder com status <code>201</code> e o produto criado.
PUT	/produtos/:id	Atualiza um produto existente. Se não existir, responder com <code>404</code> . Responder com o produto atualizado.
DELETE	/produtos/:id	Remove o produto do array. Se não existir, responder com <code>404</code> . Caso contrário, responder com status <code>204</code> ou com uma mensagem de sucesso.

Validações mínimas

- Não permitir cadastrar um produto sem `nome` ou sem `tipo` — responder com status `400` e uma mensagem clara.
- O campo `status` deve aceitar apenas os valores `disponivel`, `emprestado` ou `manutencao`.
- O `id` nunca deve ser informado pelo cliente nas operações de criação — sempre gerado pelo backend.

Dica: para gerar o próximo `id` automaticamente, uma abordagem simples é pegar o maior `id` atual do array e somar 1. Por exemplo: `Math.max(0, ...produtos.map(p => p.id)) + 1`.

4. Requisitos do Frontend

Mantenha a listagem com os filtros já existentes e adicione as operações de criação, edição e exclusão diretamente na mesma página (não é necessário criar páginas separadas). Pode usar HTML, CSS e JavaScript puro — sem frameworks.

- **Formulário de cadastro:** com campos para `nome`, `tipo`, `status` e `descricao`. Ao enviar, faz `POST` e atualiza a tabela.
- **Botão "Editar"** em cada linha da tabela: ao clicar, preenche o formulário com os dados do produto selecionado e ao confirmar realiza um `PUT`.
- **Botão "Excluir"** em cada linha: ao clicar, pede confirmação ao usuário (use `confirm()`) e realiza um `DELETE`.
- **Feedback visual:** mostrar uma mensagem de sucesso ou erro após cada operação (pode ser um `alert` ou uma `div` de mensagem).
- Após qualquer operação que altere os dados, a tabela deve ser **atualizada** automaticamente.

5. Critérios de avaliação (rubrica)

Item avaliado	Pontos
Backend — Leitura (GET por id) Rota <code>GET /produtos/:id</code> funcional, com tratamento de <code>404</code> .	0,5
Backend — Criação (POST) Rota <code>POST /produtos</code> funcional, com geração automática de <code>id</code> e validações mínimas.	0,8
Backend — Atualização (PUT) Rota <code>PUT /produtos/:id</code> funcional, com tratamento de <code>404</code> .	0,6
Backend — Exclusão (DELETE) Rota <code>DELETE /produtos/:id</code> funcional, com tratamento de <code>404</code> .	0,5
Frontend — Formulário de cadastro Permite criar novos produtos consumindo a API.	0,6
Frontend — Edição Botão de editar funcional, com pré-preenchimento do formulário.	0,5
Frontend — Exclusão Botão de excluir funcional, com confirmação ao usuário.	0,3
Organização e entrega Repositório público no GitHub, <code>README.md</code> com instruções claras de execução, código organizado e commits descritivos.	0,2
TOTAL	4,0

6. Entrega

1. Criar um repositório **público** no GitHub (ou GitLab) contendo as pastas `backend/` e `frontend/`.
2. Incluir um arquivo `README.md` com:
 - Nome dos integrantes (até 2 alunos);
 - Instruções para instalar e rodar o projeto (`bun install` e `bun run dev`);
 - Pequena descrição das rotas implementadas.
3. Enviar o **link do repositório** ao professor pelos canais combinados em aula (e-mail ou plataforma da disciplina) **até o dia 01/06/2026**.

Prazo: a entrega deve ser feita **até 01/06/2026**. Envios após essa data não serão aceitos.

Atenção: repositórios *privados* ou links que não abrem (404) **zeram a entrega**. Antes de enviar, abra o link em uma janela anônima do navegador e confirme que o conteúdo aparece.

7. Regras gerais

- **Pode ser feito sozinho ou em dupla.** Trios e grupos maiores não serão aceitos.
- Os **commits devem ser feitos pelos integrantes** da dupla — não envie um repositório com um único commit gigante de última hora.
- Cópia entre duplas, ou uso de código de terceiros sem citação, implica em **nota zero para todos os envolvidos**.
- É **permitido** consultar a documentação oficial do Express, MDN, Bun e materiais da disciplina.
- O uso de assistentes de IA é **permitido como apoio**, mas o aluno deve ser capaz de explicar qualquer trecho do código entregue se solicitado.

8. Dicas para um bom resultado

- Comece pelo backend: implemente e teste cada rota com `curl`, Postman ou Insomnia antes de mexer no frontend.
- Lembre-se do `app.use(express.json())` — sem ele, o backend não consegue ler o corpo das requisições `POST` / `PUT`.
- No `fetch` do frontend para `POST` / `PUT`, não esqueça do header `Content-Type: application/json` e do `JSON.stringify` no corpo.
- Faça commits pequenos a cada funcionalidade concluída — facilita a correção e demonstra evolução do trabalho.

Bom trabalho! Em caso de dúvidas, procurem o professor durante as aulas ou pelos canais oficiais da disciplina.

Prof. Ivan Luiz Pedroso Pires · DSW 2026/1 · UNEMAT